PLAYWRIGHT TYPESCRIPT · 2026 EDITION

This vs That

Rehearsal Cards

Cover the answer, say it out loud. The comparison questions from your Foundation kit, drilled until they come out clean.

by Sreenidhi Rajakrishnan · Automation Architect

How to use these rehearsal cards

> **Start with what this is.** These are not new questions. Every pair below is already answered in your Foundation kit, the 645 page Ultimate Q&A, and condensed again in the 169 page Quick Revision. Six of them are word for word Foundation questions. That overlap is on purpose.
>
> **What is different is the format.** The Foundation kit teaches you the answer. This one trains you to say it. Each pair is one tight spoken paragraph, short enough to cover with your hand and recite from memory, in the words you would actually use in the room. Reading an answer and saying it under pressure are two different skills, and only the second one gets scored.
>
> **So treat it as a drill, not as reading.** Cover the answer, say it out loud, then check yourself. Run the whole set the night before an interview, and again in the last ten minutes. Repetition is the point.

Interviewers lean on the "what is the difference between X and Y" question because it separates people who have read about Playwright from people who have used it. A vague answer sounds memorised. A sharp one, cause first then contrast, sounds lived-in. It is also usually the first thing you are asked, which is why these are the answers worth having on your tongue rather than in your notes.

The strongest answer is always the same shape: name the real difference in one line, then say which one you reach for, and why. Where a comparison has moved on since the tutorials were written, the card says so, so that you are not repeating something an interviewer will correct.

Architecture and tooling

Playwright vs Selenium

Selenium drives the browser through the W3C WebDriver protocol, and since Selenium 4 it can also open a WebDriver BiDi connection for events, logs and network work, so the old line about it being blind to the browser no longer holds. Playwright keeps one persistent connection of its own, CDP for Chromium and its own protocol for Firefox and WebKit, which is what makes auto-waiting and tracing straightforward. Selenium is the open standard with the widest language, browser and grid reach and is still the right answer for a large existing estate. Playwright gives you one modern API with the runner, fixtures and traces in the box.

Playwright vs Cypress

Do not repeat the old cross-origin line: cy.origin() has handled cross-origin flows since 2022, so that is not the difference any more. The real one is the runner model. Cypress runs your test code inside the browser beside the app, which is why its time-travel debugging is so good. Playwright drives the browser from a separate process outside it, which is what lets it work with more than one tab or window in a single test, something Cypress does not do by design. The other honest difference is reach: Cypress is JavaScript and TypeScript, Playwright also ships Python, Java and .NET. Cypress suits a front-end team living in one app; Playwright suits multi-tab, multi-browser and mixed-language work.

Playwright vs Puppeteer

Puppeteer is a browser automation library, Chrome first, with Firefox support through WebDriver BiDi. It is a library rather than a test framework: no runner, no fixtures, no web-first assertions, no trace viewer. Playwright came from the same lineage and added all of those, plus WebKit. To script a browser, Puppeteer is fine and lighter. To build a test suite, Playwright is the framework.

Locators and elements

Locator vs ElementHandle

An ElementHandle points at one element found at one moment, so it goes stale if the DOM changes. A Locator is a description re-resolved every time you use it, and it auto-waits. Prefer the Locator; reach for a handle only when the Locator API cannot cover the case.

page.locator() vs page.$ / $$

The dollar methods return handles immediately and do not auto-wait, which is exactly why they are discouraged. locator() returns a lazy, auto-waiting Locator. Use locator(); treat the dollar methods as legacy.

getByRole vs CSS / XPath

The getBy locators find elements the way a user or assistive technology would, by role, label or visible text, so they survive markup changes. CSS and XPath tie the test to page structure and break on refactors. Lead with getByRole and getByTestId; drop to CSS only when you must.

fill() vs pressSequentially()

fill() sets a field's value in one go, the fast default for forms. pressSequentially(), the replacement for the old type(), sends key by key, which is only needed when the field reacts to each keystroke, such as an autocomplete.

check() vs click()

click() toggles whatever is present. check() asserts the intent: it ensures the box ends up checked and verifies the state afterwards. For checkboxes and radios, check() and uncheck() are the safer, declarative choice.

Assertions

expect vs expect.soft

A normal expect stops the test the moment it fails. expect.soft records the failure but lets the test continue, so one run can report several problems at once. Use soft assertions to surface everything on a page, then a hard check at the end.

Web-first assertions vs Generic assertions

A web-first assertion such as toBeVisible retries until it passes or times out, absorbing normal page delay. A generic assertion checks once, immediately. Use web-first assertions for anything touching the page, and generic ones only for plain values already in hand.

toBeVisible vs toBeAttached

toBeAttached only checks the element is in the DOM. toBeVisible checks it is in the DOM and actually shown, so visible implies attached. toBeInViewport is a separate question rather than a stricter one: it asks whether the box intersects the viewport and does not require the element to be visible at all, so something set to visibility hidden in the middle of the screen passes toBeInViewport and fails toBeVisible. Choose the assertion that matches the question you are asking.

Waiting and loading

Auto-waiting vs Explicit waits

Before an action Playwright waits for that action's actionability checks, and the detail worth knowing is that they differ by action. A click waits for visible, stable, receives events and enabled. A fill waits for visible, enabled and editable only, with no stability or hit-target check, which is why a cookie banner blocks a click and not a fill. Some methods run no checks at all, including focus, press and dispatchEvent. That still removes most of the sleeps scattered through older suites. You add an explicit wait only for something Playwright cannot infer, such as a specific network response.

waitForTimeout vs Locator auto-wait

waitForTimeout is a fixed sleep and a last resort, because it is either too short and flaky or too long and slow. Locator auto-wait waits for the exact element you are about to use, and a web-first assertion waits for the state you care about. waitForLoadState waits for a page event, which is a blunter instrument and rarely what you actually need. Prefer auto-wait and assertions, then a specific response, and avoid fixed sleeps.

domcontentloaded vs load and networkidle

domcontentloaded fires when the HTML is parsed and load when images and sub-resources have finished. networkidle waits for the network to go quiet, and Playwright now discourages it: on an app that polls, streams or holds a socket open the network never goes quiet, so the wait either hangs or clears for the wrong reason. Saying that out loud lands well, because plenty of candidates still recommend it. In practice you rarely wait on a load state at all. Wait for the thing you actually need: a web-first assertion such as expect(locator).toBeVisible(), a specific call with page.waitForResponse, or a state you can check directly.

Structure and execution

Fixtures vs beforeEach / afterEach

Hooks run setup and teardown in order and share state through variables. Fixtures are named, reusable pieces of setup that Playwright creates only when a test asks for one, tears down automatically, and can build on each other. Fixtures scale to a real framework; hooks are fine for a single file.

browserContext vs page

A browser is one launched engine. A context is an isolated session inside it, with its own cookies and storage, like a fresh incognito window. A page is a single tab. Two contexts in the same browser share the browser process and nothing else, no cookies and no storage, and that is the whole point: give each test its own context for clean isolation without relaunching the browser. Watch this one in the room, because saying a new context inherits the browser's login is a common slip and an easy correction for the interviewer.

Headed vs Headless

Headless runs with no visible window, which is faster and what CI uses. Headed shows the browser, which you use while writing or debugging. The behaviour is identical; it is only about whether you can watch it.

Workers vs Shards

Workers are parallel processes on one machine, set by the runner. Shards split the whole suite across several machines in CI, each running a slice. Workers scale you up on one box; shards scale you out across many.

Network, debugging and timeouts

route: fulfill vs route: abort

Inside page.route you decide what happens to a request. fulfill answers it with your own mock response, continuing with changes lets it reach the server with an altered body or headers, and abort blocks it entirely. One interception, three levers.

Trace Viewer vs UI mode

page.pause() opens the Inspector live so you can step through a running test. The Trace Viewer is an after-the-fact recording of a finished run, with snapshots, network and console, ideal for a CI failure you cannot reproduce. UI mode is an interactive runner for writing and re-running tests locally.

expect timeout vs test timeout

The expect timeout bounds how long a single web-first assertion retries. Action and navigation timeouts bound one click or one page load. The test timeout is the ceiling for the whole test. They stack, so name which of the three you mean before raising any numbers.